2

Background on graph neural networks

This section provides the indispensable background notions regarding graph neural networks (GNNs). After formally defining graphs, the section presents the message passing algorithm, which is the basic procedure behind most of the GNN architectures from the literature. Then, it introduces two of the pioneer GNN models, namely, graph convolutional network (GCN) and graph attention network (GAT), whose procedures inspired many graph-based recommendation approaches. Finally, it describes the task of link prediction in graph representation learning, for whom recommendation represents one of the possible instantiations.

2.1 Basic notions about graphs

This section begins with some basic notions about graphs as data structures. After a formal definition of what a graph is, along with its multiple variations, we formalize the adjacency matrix and node features, which represent the main components to define a graph.

2.1.1 Definition of graph

We define a graph through the set of nodes $\mathcal{V}$ and edges $\mathcal{E}$ connecting such nodes, namely, $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. Specifically, an edge between node $v \in \mathcal{V}$ and $w \in \mathcal{V}$ exists if the two nodes are connected in the graph. When edges in a graph come with a direction (e.g., node v is connected to node w but not vice versa), the graph is *directed*; otherwise, if the edge direction is not defined (e.g., v is connected to w and also the vice versa holds) the graph is *undirected*.

Moreover, nodes in a graph may belong to disjoint partitions, namely, $\mathcal{V} = \mathcal{V}_1 \cup \mathcal{V}_2, \ldots, \cup \mathcal{V}_p$, having $\mathcal{V}_i \cap \mathcal{V}_j = \emptyset \, , \forall i \neq j$. This type of graph, named *heterogeneous*, has edges that can connect nodes from different partitions following a specific rationale. A special case of heterogeneous graphs is the *multipartite* one, a graph where edges only connect nodes from different partitions. Among multipartite graphs, *bipartite* graphs contain nodes that belong to two different partitions, and nodes from one partition can only be connected to nodes from the other partition.

2.1.2 Adjacency matrix and node features

An *adjacency matrix* represents the graph $\mathcal{G}$ into matrix format. By assigning each node in the graph an ordering (i.e., nodes are represented by specific rows and columns in the matrix), the adjacency matrix $\mathbf{A} \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{V}|}$ is designed such that $A_{vw} = a$ if there exists an edge connecting v and w, 0 otherwise. In a directed graph, $A_{vw} \neq A_{wv}$ in general, while in an undirected graph, $\mathbf{A}$ is symmetric and $A_{vw} = A_{wv}$.

While the entry $A_{vw} = a$ is a real-valued number that represents the weight of the edge connecting the nodes v and w, it is common practice (for the sake of simplicity) to consider binary adjacency matrices where $A_{vw} = \{0, 1\}$ and $A_{vw} = 1$ if there exists an edge between the two nodes, 0 otherwise. In multipartite graphs there exist entire portions of the adjacency matrix whose entries are zero, as in such a family of graphs some node partitions are not connected to one another.

Moreover, for the training of graph neural networks (see later), the adjacency matrix is usually transformed to obtain the symmetric

normalized adjacency matrix (i.e., $\tilde{\mathbf{A}}$):

$$\tilde{\mathbf{A}} = \mathbf{D}^{-\frac{1}{2}}\mathbf{A}\mathbf{D}^{-\frac{1}{2}}, \tag{2.1}$$

where the purpose of this normalization is to flatten the weight importance of all nodes in the graph based upon their degree to tackle possible instabilities during the optimization of the graph neural network.

In addition, another important concept is the *neighborhood* of a node v, defined as the set of nodes which are connected to v through one edge. Let $\mathcal{N}_v = \{w \in \mathcal{V} \mid A_{vw} = 1\}$ be the neighborhood of v.

Finally, nodes in a graph may be associated with attributes or features describing them. Generally, such features are formally represented through a real-valued matrix $\mathbf{X} \in \mathbb{R}^{|\mathcal{V}| \times F}$ where F is the dimensionality of the feature matrix.

2.2 The message passing algorithm

Independently on the graph learning strategy adopted, the most atomic building block of any graph neural network architecture lies in the *message passing* algorithm (Gilmer *et al.*, 2017).

In its most generalized version, such a procedure works by *updating* the latent representation of each node (defined as *ego* node in this context) through the information conveyed by the nodes directly connected to the ego one (defined as *neighbor* nodes). The information from the neighborhood is usually referred to as *messages*, which are *aggregated* and used to update the representation of the ego node. Finally, the message passing schema is iteratively applied for L layers, and the various node representations are eventually *combined* to obtain a unique representation for each node. In the following, a formal definition for each step, namely, *message aggregation*, *node embedding update*, and *layer combination*, is presented (Hamilton, 2020).

2.2.1 Message aggregation

Let $\mathbf{X}_v^{(l-1)} \in \mathbb{R}^{F_{l-1}}$ be the representation of node $v \in \mathcal{V}$ at layer $l-1$, with $1 \leq l \leq L$, and F_{l-1} the embedding dimension at layer $l-1$. Note that we have $\mathbf{X}_v^{(0)} = \mathbf{X}_v$. We obtain the aggregated messages from the

neighbor nodes of v through the following:

$$\mathbf{M}_v^{(l-1)} = \text{Aggregate}^{(l)}(\{\mathbf{X}_w^{(l-1)} \mid w \in \mathcal{N}_v\}), \tag{2.2}$$

where $\mathbf{M}_v^{(l-1)} \in \mathbb{R}^{F_{l-1}}$ is the aggregated message from the neighbor nodes of v at layer $l-1$ and $\text{Aggregate}^{(l)}(\cdot)$ is the aggregation function over the neighbor nodes of v. For instance, a popular choice for the $\text{Aggregate}^{(l)}(\cdot)$ function is the element-wise addition:

$$\mathbf{M}_v^{(l-1)} = \sum_{w \in \mathcal{N}_v} A_{vw} \mathbf{X}_w^{(l-1)}, \tag{2.3}$$

where A_{vw} corresponds to the weight of the edge between v and w (if the nodes are connected). As stated above, the adjacency matrix may be normalized into the symmetric normalized adjacency matrix:

$$\mathbf{M}_v^{(l-1)} = \sum_{w \in \mathcal{N}_v} \frac{A_{vw}}{\sqrt{|\mathcal{N}_v||\mathcal{N}_w|}} \mathbf{X}_w^{(l-1)}, \tag{2.4}$$

where the denominator involves the node degrees of the ego node and each neighbor node.

2.2.2 Node embedding update

Once the aggregated message from the neighbor nodes has been calculated, the node embedding can be updated to obtain the representation at layer l:

$$\mathbf{X}_v^{(l)} = \text{Update}^{(l)}(\mathbf{X}_v^{(l-1)}, \mathbf{M}_v^{(l-1)}), \tag{2.5}$$

where $\mathbf{X}_v^{(l)} \in \mathbb{R}^{F_l}$ is the updated embedding representation of node v at layer l, $\text{Update}^{(l)}(\cdot)$ is the update function for the ego node at layer l, and $\mathbf{X}_v^{(l-1)}$ is usually included to leverage the current representation of the ego node into the message passing formulation. For instance, a popular choice to design the $\text{Update}^{(l)}(\cdot)$ function is the feature transformation of the two inputs:

$$\mathbf{X}_v^{(l)} = \text{Activate}(\mathbf{W}_{\text{ego}}^{(l)} \mathbf{X}_v^{(l-1)} + \mathbf{W}_{\text{neigh}}^{(l)} \mathbf{M}_v^{(l-1)}), \tag{2.6}$$

where $\text{Activate}(\cdot)$ is the activation, such as the Sigmoid or ReLU, while $\mathbf{W}_{\text{ego}}^{(l)} \in \mathbb{R}^{F_{l-1} \times F_l}$ and $\mathbf{W}_{\text{neigh}}^{(l)} \in \mathbb{R}^{F_{l-1} \times F_l}$ are the weights referring to the ego node and the message aggregated through the $\text{Aggregate}^{(l)}(\cdot)$ function, respectively.

2.2.3 Layer combination

The final stage is the combination of all node embedding representations obtained in each layer l. The layer combination is obtained through:

$$\hat{\mathbf{X}}_v = \text{Combine}(\{\mathbf{X}_v^{(0)}, \mathbf{X}_v^{(1)}, \ldots, \mathbf{X}_v^{(l)}, \ldots, \mathbf{X}_v^{(L)}\}), \tag{2.7}$$

where Combine$(\cdot)$ is defined through different operations, such as the element-wise addition, the average, the concatenation, or it is simply obtained as the last node update from layer L.

2.2.4 Matrix format and self-loops

The formulations provided above for the message passing algorithm are expressed at node level. However, in most of the cases, the message passing algorithm can be expressed into a more compact matrix format which comprises all graph nodes at once:

$$\mathbf{X}^{(l)} = \text{Activate}(\mathbf{X}^{(l-1)}\mathbf{W}_{\text{ego}}^{(l)} + \mathbf{A}\mathbf{X}^{(l-1)}\mathbf{W}_{\text{neigh}}^{(l)}), \tag{2.8}$$

where $\mathbf{X}^{(l-1)} \in \mathbb{R}^{|\mathcal{V}| \times F_{l-1}}$ includes all node embeddings. Generally, the Update$^{(l)}(\cdot)$ function can be simplified by adding self-loops in the adjacency matrix, namely, the ego node is added to the set of neighbor nodes during the message aggregation:

$$\mathbf{X}^{(l)} = \text{Activate}((\mathbf{A} + \mathbf{I})\mathbf{X}^{(l-1)}\mathbf{W}^{(l)}), \tag{2.9}$$

where $\mathbf{A} + \mathbf{I}$ is used to add self-loops to the adjacency matrix, and the presence of a single $\mathbf{W}^{(l)}$ matrix suggests $\mathbf{W}_{\text{ego}}^{(l)}$ and $\mathbf{W}_{\text{neigh}}^{(l)}$ are shared into one matrix.

2.3 Popular graph neural network architectures

The literature recognizes different graph neural network architectures based on the specific strategies and operations adopted for message passing. In the following, we present and formalize the most popular ones, namely, graph convolutional network (GCN) and graph attention network (GAT), whose architectures inspired many graph-based recommender systems.

2.3.1 Graph convolutional network

The graph convolutional network architecture (GCN) proposed by Kipf and Welling (2017) is one of the pioneer works in graph neural networks. The layer is defined as:

$$\mathbf{X}^{(l)} = \text{ReLU}(\tilde{\mathbf{A}}\mathbf{X}^{(l-1)}\mathbf{W}^{(l)}), \tag{2.10}$$

where $\tilde{\mathbf{A}} = (\mathbf{D}+\mathbf{I})^{-\frac{1}{2}}(\mathbf{A}+\mathbf{I})(\mathbf{D}+\mathbf{I})^{-\frac{1}{2}}$ is the symmetric normalized adjacency matrix with self-loops obtained through the identity matrix $\mathbf{I} \in \mathbb{R}^{|\mathcal{V}|\times|\mathcal{V}|}$.

2.3.2 Graph attention network

The graph attention network architecture (GAT) was first introduced by the work of Velickovic *et al.* (2018) and it leverages attention mechanisms to weight the relative importance of each neighbor node to its corresponding ego node. Specifically, the message passing schema of GAT is defined as:

$$\mathbf{X}^{(l)} = \text{ReLU}(\hat{\mathbf{A}}\mathbf{X}^{(l-1)}\mathbf{W}^{(l)}), \tag{2.11}$$

where $\hat{\mathbf{A}} \in \mathbb{R}^{|\mathcal{V}|\times|\mathcal{V}|}$ is the weight matrix obtained via attention mechanisms. Specifically, by considering the ego node $v \in \mathcal{V}$ along with its neighbor nodes $\forall w \in \mathcal{N}_v$, the attention mechanism is formulated as:

$$\hat{A}_{vw} = \frac{\exp(\text{LeakyReLU}(\mathbf{T}[\mathbf{W}\mathbf{X}_v^{(l-1)}||\mathbf{W}\mathbf{X}_w^{(l-1)}]))}{\sum_{w\in\mathcal{N}_v}\exp(\text{LeakyReLU}(\mathbf{T}[\mathbf{W}\mathbf{X}_v^{(l-1)}||\mathbf{W}\mathbf{X}_w^{(l-1)}]))}, \tag{2.12}$$

where $\exp(\cdot)$ is the exponential function, $\mathbf{T} \in \mathbb{R}^{1\times 2F_{l-1}}$ is the projection matrix, and $||$ is the concatenation operation.

2.4 The link prediction task

This last part is devoted to presenting a popular task in graph representation learning, namely, link prediction, since recommendation represents one of its possible instantiations.

As for other tasks in graph representation learning (e.g., node and graph classification), it is possible to formalize link prediction through

the *encoder-decoder* framework (Chami *et al.*, 2022). In its most generic version, an *encoder* function maps the adjacency matrix and initial node features of a graph to an embedded representation of the nodes:

$$\text{ENCODER: } \mathbb{R}^{|\mathcal{V}|\times|\mathcal{V}|} \times \mathbb{R}^{|\mathcal{V}|\times F_0} \to \mathbb{R}^{|\mathcal{V}|\times F_L}. \tag{2.13}$$

From the above, the *encoder* function is formulated as any message passing procedure repeated for L propagation layers. Regarding the *decoder* function, it takes the encoded representation of the nodes as input, and produces some outputs which depend on the considered task.

Link prediction is the task of predicting if an edge exists between any two nodes of a graph. Specifically, we consider a subset of graph edges on which the graph learning model will be trained (i.e., the training set), and mask the remaining ones (i.e., the test set). The graph learning model will be trained to correctly predict the presence and absence of an edge in the training set, to eventually be evaluated on the predictions for the unseen links in the test set. Thus, the decoder function in link prediction is defined as:

$$\text{DECODER: } \mathbb{R}^{|\mathcal{V}|\times F_L} \to \mathbb{R}^{|\mathcal{V}|\times|\mathcal{V}|}, \tag{2.14}$$

where we aim to reconstruct the whole adjacency matrix and make predictions on the existence of edges in the graph. Practically speaking, the decoder is generally designed as a function computing all similarities between any pairs of learned node embeddings, for instance through their dot product:

$$\hat{\mathbf{A}} = \hat{\mathbf{X}}^\top \hat{\mathbf{X}}. \tag{2.15}$$